Ygeiopolis Healthcare System - Final Report
Academic year 2025-2026

## 1. Project scope

This repository implements a relational database for the General Hospital "Ygeiopolis". The schema covers departments, personnel, shifts, emergency visits, patients, hospitalizations, ICD-10 diagnoses, KEN costing, lab tests, procedures, prescriptions, drug active substances, allergies, patient evaluations, and image metadata.

The implementation targets MySQL/MariaDB with InnoDB tables. It avoids SQL ENUM, arrays, JSON, and XML. Controlled values are represented with VARCHAR columns and CHECK constraints.

## 2. Submitted files

The final submission includes README.md, diagram PDFs, docs/report.pdf, sql/install.sql, sql/load.sql, sql/validation.sql, sql/Q01.sql through sql/Q15.sql, sql/Q01_out.txt through sql/Q15_out.txt, scripts/generate_data.py, the CSV files under data/reference and data/generated, and the optional Streamlit demo files.

## 3. Schema design

Common personnel data is separated from role-specific doctor, nurse, and administrative_staff tables. Hospital structure is represented with department, bed, and operating_place. Patient care is represented with emergency_visit and hospitalization. Clinical events during hospitalization are represented with lab_test, procedure_event, procedure_participant, and prescription.

Reference data is normalized in ICD-10, KEN, procedure, lab-test, drug, and active-substance tables. The drug and active-substance data is loaded from the official EMA Article 57 reference data.

## 4. Integrity rules

Primary keys, foreign keys, unique constraints, and CHECK constraints are used throughout the schema. Triggers enforce rules that need cross-row checks: doctor hierarchy restrictions, shift limits and coverage, allergy prevention for prescriptions, procedure room and personnel overlap prevention, procedure place type validation, and automatic hospitalization cost calculation from KEN base cost, mean duration, and extra daily cost.

## 5. Data loading and validation

The repository includes portable CSV files under data/reference and data/generated. The database can be recreated from the repository root with:

```
mysql -u root < sql/install.sql
mysql --local-infile=1 -u root < sql/load.sql
mysql -u root < sql/validation.sql
```

The validation script prints row counts and runs problem-detection queries. After the final load, the problem-detection result sets are empty.

## 6. Query set

The submitted sql/Q01.sql through sql/Q15.sql files are the final assignment queries. Their outputs are stored in sql/Q01_out.txt through sql/Q15_out.txt.

Some queries use CTEs where they make the result easier to explain. Q05 counts

young chief surgeons first and then keeps the maximum count so ties are returned. Q06 computes the selected patient's evaluation average once and reuses it for each hospitalization row. Q09 computes total hospitalization days per patient and year before checking equal totals. Q13 uses a recursive CTE to walk the doctor-supervisor hierarchy. Q14 counts yearly admissions per ICD-10 diagnosis before comparing consecutive years. Q15 computes emergency-level totals once and uses them as denominators for final percentages.

## 7. Index and EXPLAIN checks

Indexes were added for the most important filtering and join paths. Q04 uses hospitalization_doctor through doctor_amka; the optimal lookup is the foreign-key-created index fk_hosp_doctor_doctor. The evaluation lookup uses idx_evaluation_evaluation on hosp_id. Q06 benefits from idx_hosp_patient_admission_q6 because the query filters by patient_amka and orders by admission_ts.

During testing, EXPLAIN ANALYZE was used with FORCE INDEX variants to compare optimal and non-optimal plans. The final submitted query files do not contain EXPLAIN ANALYZE; they return the required result tables.

## 8. Application note

The Streamlit app is optional. It is a read-only demonstration dashboard for loading predefined Q01-Q15 queries and inspecting hospital operational data. The database submission is complete without running the app.